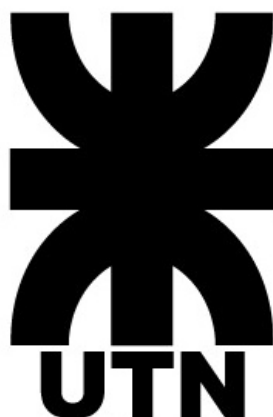


UNIVERSIDAD TECNOLÓGICA NACIONAL

FACULTAD REGIONAL CÓRDOBA



INGENIERÍA EN SISTEMAS DE INFORMACIÓN

ASEGURAMIENTO DE CALIDAD DE PROCESO Y DE

PRODUCTO

TRABAJO PRÁCTICO: DESARROLLO DIRIGIDO POR PRUEBAS

(TDD)

Curso: 4K4

Docentes:

- Ing. Laura Covaro
- Ing. Constanza Garnero
- Salvador Barbera

GRUPO 10

Integrantes:

- Carranza Duobaitis, Candelaria (402600)
- Martínez Agliozzo, Luz María (94415)
- Llabot, Ignacio Martín (403083)
- Salvatico, Francisco (403815)
- Sana, Fabrizio (98161)
- Schurrer, Gabriel Nicolas (95046)
- Maldonado, Franco Gabriel (92123)
- Nobile, Bruno Nicolas (92098)
- Marti, Eliazar Alexis (403419)
- Cavina Pellis, Manuel (95648)
- Palomeque Galindo, Matías Ezequiel (94482)

Documento de Justificación de Decisiones de Diseño

1. Herramientas y Estándares de Código

Para la materialización de la solución técnica, enfocada en la User Story "Inscribirme a actividad" del sistema EcoHarmony Park, se seleccionaron herramientas que garantizan la robustez, mantenibilidad y escalabilidad a nivel corporativo:

- **Lenguaje y Framework:** Java fue adoptado como lenguaje base, utilizando JUnit 5 (jupiter-engine) para la orquestación de pruebas unitarias y Maven como gestor centralizado del ciclo de vida y las dependencias del proyecto.
- **Estilos de Codificación y Nomenclatura:**
 - a. Se aplica *PascalCase* para las Clases (ej: *InscripcionService*) y *SCREAMING_SNAKE_CASE* para las Constantes.
 - b. Para Métodos y Variables, se utiliza *camelCase*. **El Ubiquitous Language en español se aplica estrictamente a la denominación de métodos** (ej: *inscribir()*) y variables, garantizando la trazabilidad semántica directa con las reglas de negocio y los Criterios de Aceptación.
 - c. **Estándar de Longitud de Línea:** Se impone un límite estricto de **120 caracteres** por línea de código, optimizando la legibilidad en entornos de revisión y la gestión del control de versiones.
- **Convención de Nombres para Tests:** Se implementó una convención de nomenclatura explícita para diferenciar la intencionalidad de la prueba:
 - a. *testFallaPor[MOTIVO]* (Para escenarios de excepción, ej: *testFallaPorDniConPuntos*).
 - b. *testInscripcionExitosa[MOTIVO]* (Para validaciones funcionales positivas, ej: *testInscripcionExitosaAlLimiteDelCupo*).

2. Mapeo de Casos de Prueba (Aplicación de Técnicas de Caja Negra)

Con el fin de asegurar una cobertura funcional exhaustiva, el conjunto de pruebas fue ampliado sistemáticamente a más de 20 escenarios mediante la aplicación de Partición de Clases de Equivalencia (PCE) y Análisis de Valores Límite (AVL). La implementación de la suite de pruebas se basa en una **estrategia de métodos de prueba atómicos, altamente descriptivos y aislados**. Esta decisión de diseño facilita la **trazabilidad exacta del error** a un único caso de negocio o restricción validada, a diferencia de la complejidad inherente a la depuración de pruebas con múltiples parámetros. La suite de pruebas abarca las siguientes categorías de validación:

- **Validaciones de Datos del Visitante (PCE y AVL)**
 - **Nombres y Apellidos:** Inyección de valores límite (longitud excedida) y clases inválidas (cadena vacía o nula) (Métodos de Test: *testFallaPorNombreVacio*, *testFallaPorApellidoExcedido*).
 - **DNI:** Pruebas de casos límite (números negativos, longitud incorrecta) y clases inválidas (caracteres alfanuméricos o nulos) (Método de Test: *testFallaPorDniConPuntos*).

- **Edad:** Aplicación de Valores Límite (AVL) en los rangos inferiores (Ej.: -1, 0, 1) y superiores (Ej.: 150) para asegurar la lógica de restricción por edad de la actividad. (Método de Test: *testFallaPorEdadFueraDeRango*).
- **Validaciones de Cupos y Personas (AVL)**
 - **Cupo al Límite:** Verificación de la inscripción exitosa en la capacidad máxima (Valor Límite Superior) (Método de Test: *testInscripcionExitosaAlLimiteDelCupo*).
 - **Cupo Excedido:** Prueba de la falla al intentar inscribir a una persona más de la capacidad total (Valor Límite Superior + 1). (Método de Test: *testFallaPorFaltaDeCupo*).
 - **Cantidad de Personas:** Se prueban valores inválidos de cantidad de personas a inscribir (Ej.: 0, -1, nulo) para asegurar la falla inmediata. (Método de Test: *testFallaPorCantidadCeroONegativa*).
- **Validaciones de Talles de Vestimenta (PCE)**
 - **Talles Válidos:** Inclusión de todas las clases de equivalencia aceptadas ("S", "M", "L", "XL"). (Método de Test: *testInscripcionExitosaConTalleRequerido*).
 - **Talles Inválidos:** Pruebas con talles inexistentes (Ej.: "Z", "XXL"), formatos no esperados (Ej.: números, caracteres especiales), o talle nulo cuando es requerido. (Método de Test: *testFallaPorTalleInvalido*).
- **Validaciones de Actividades y Horarios (PCE)**
 - **Actividades:** Prueba de inscripciones con referencias a Actividades inexistentes o nulas. (Método de Test: *testFallaPorActividadInexistente*).
 - **Horarios:** Pruebas con formatos de hora inválidos (Ej.: "texto:00", "25:00") o que se encuentran fuera del horario operativo del parque. (Método de Test: *testFallaPorHorarioDeParqueCerrado*, *testFallaPorFormatoDeHoraInvalido*).
 - **Términos y Condiciones:** Caso de falla explícito por el rechazo de los términos y condiciones de la inscripción. (Método de Test: *testFallaPorTerminosNoAceptados*).

3. Aplicación del Ciclo TDD (Red-Green-Refactor)

La metodología de desarrollo adoptada fue estrictamente iterativa, asegurando la calidad del código mediante la adhesión al ciclo TDD, abordando un criterio de aceptación por iteración:

1. **Fase RED:** Definición del contrato funcional mediante la escritura de un caso de prueba unitario que, por diseño, fallaba o no compilaba debido a la ausencia de la lógica de negocio requerida.
2. **Fase GREEN:** Implementación de la lógica mínima, y necesaria, para satisfacer la condición de la prueba y lograr su ejecución exitosa.
3. **Fase REFACTOR:** Optimización y pulido del código, extrayendo lógica a componentes privados y mejorando la legibilidad, manteniendo la suite de pruebas en estado "verde" para confirmar la inexistencia de regresiones.

4. Justificación de Decisiones de Diseño

Se fundamentan las siguientes decisiones arquitectónicas tomadas para garantizar la robustez y mantenibilidad del sistema:

- **Gestión de Errores (Fail-Fast):** Se optó por el uso de excepciones personalizadas (*RuntimeException*) para gestionar flujos de error, en lugar de retornar valores booleanos o códigos de error. Esta decisión promueve un corte abrupto y limpio del hilo de ejecución, implementando un patrón de "Fail-Fast" en la capa de servicio.
- **Estructura de Pruebas (Patrón AAA):** Todos los métodos de prueba se adhirieron al estándar estructural **Arrange, Act, Assert (AAA)**. Esta formalidad garantiza una legibilidad uniforme y aísla de manera inequívoca la inicialización del contexto (Arrange), la ejecución de la acción bajo prueba (Act), y la verificación de los resultados esperados (Assert).
- **Encapsulamiento y Responsabilidades:** Se definieron entidades de dominio fuertes como Visitante y Actividad. Siguiendo el principio de Information Expert, la lógica de decremento de disponibilidad se delegó a la propia entidad mediante el método `actividad.restarCupo()`, protegiendo la integridad de los datos.
- **Aislamiento de Dependencias:** La arquitectura aísla componentes volátiles como el envío de correos electrónicos y la pasarela de pagos (Mercado Pago). Esta abstracción permite la futura inyección de Mocks, garantizando que las pruebas unitarias se ejecuten de forma rápida y sin efectos secundarios externos.

Integración de TDD y Técnicas de Caja Negra: Robustez de la Cobertura

La implementación del flujo de trabajo TDD fue rigurosamente complementada con técnicas de diseño de pruebas de Caja Negra (PCE y AVL). TDD define el marco de trabajo (primero el test, luego la implementación), mientras que las técnicas de Caja Negra fueron esenciales para la identificación sistemática de condiciones externas y la modelización de los escenarios críticos de la User Story.

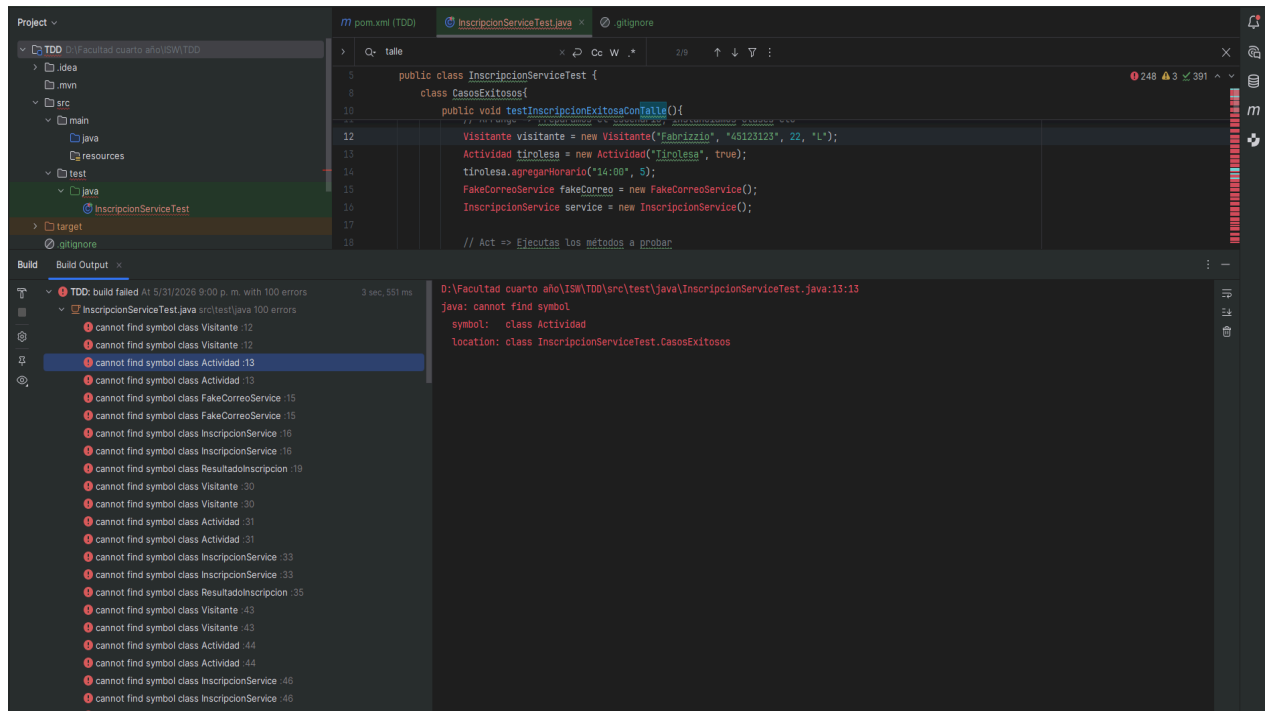
Esta sinergia no solo garantizó una cobertura funcional exhaustiva, sino que concentró el esfuerzo de validación en los comportamientos de borde y las restricciones críticas del sistema. El elevado número de escenarios de prueba (más de 20 casos) se gestionó a través de **métodos de prueba unitarios individuales, descriptivos y auto-contenidos**, lo cual es fundamental para el mantenimiento del código, ya que cualquier fallo se mapea directamente a una restricción de negocio específica, optimizando la identificación y resolución de defectos.

Aclaración: Se incluye link al repositorio con el código y archivo de Google Sheets con casos de prueba y clases de equivalencia:

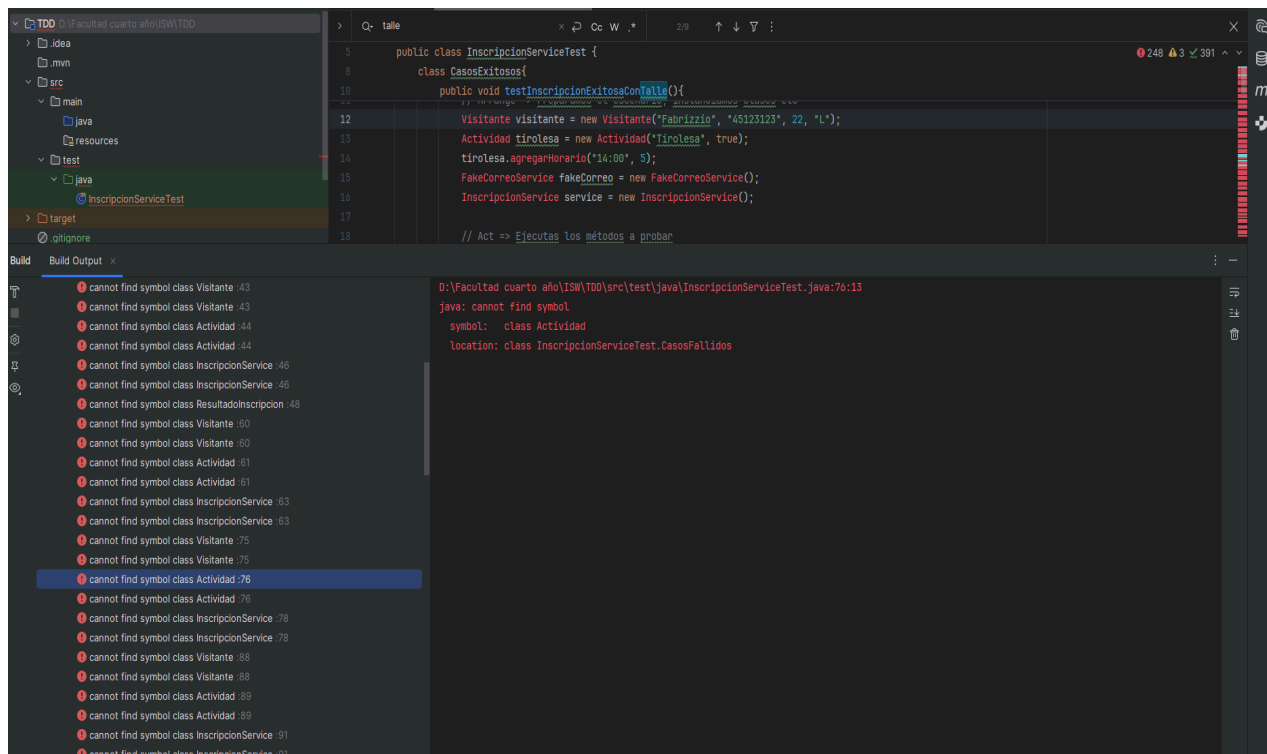
<https://github.com/fabrizzio178/utn-frc-isw-4K4-2026-G10/tree/main/CodigoFuente>

 Casos de prueba - Inscribirme a actividad

Ejemplo de RED para casos exitosos:



Ejemplos de RED para casos inválidos:



Ejemplo de GREEN para casos exitosos:

The screenshot displays a test runner interface. On the left, a tree view shows the test hierarchy: 'IncripcionServiceTest' (52 ms) is expanded, showing 'CasosFallidos' (51 ms) and 'CasosExitosos' (1 ms). Under 'CasosExitosos', three tests are listed with green checkmarks: 'testIncripcionExitosaAlLimiteDe', 'testIncripcionExitosaConTalle()', and 'testIncripcionExitosaSafariSinTalle()'. On the right, a summary bar indicates '30 tests passed' and '30 tests total, 52 ms'. Below this, a terminal window shows the command '"C:\Program Files\Java\jdk-21\bin\java.exe" ...' and the message 'Process finished with exit code 0'.

Test Name	Duration
IncripcionServiceTest	52 ms
> CasosFallidos	51 ms
✓ CasosExitosos	1 ms
✓ testIncripcionExitosaAlLimiteDe	1 ms
✓ testIncripcionExitosaConTalle()	
✓ testIncripcionExitosaSafariSinTalle()	

✓ 30 tests passed 30 tests total, 52 ms

```
"C:\Program Files\Java\jdk-21\bin\java.exe" ...  
  
Process finished with exit code 0
```

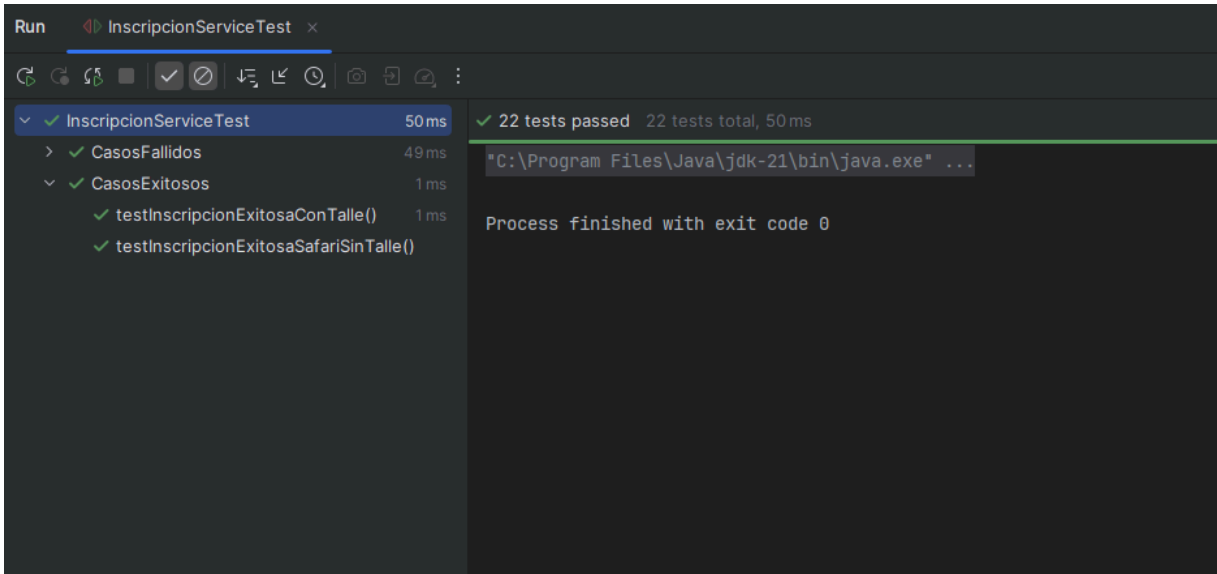
Ejemplo de GREEN para casos fallidos:

The screenshot displays a test runner interface. On the left, a tree view shows the test hierarchy: 'CasosFallidos' (51 ms) is expanded, listing 18 tests, all with green checkmarks. On the right, a summary bar indicates '30 tests passed' and '30 tests total, 52 ms'. The terminal window is empty.

Test Name	Duration
CasosFallidos	51 ms
✓ testFallaPorHorarioFueraDeF	21 ms
✓ testFallaPorCantidadCeroPers	1 ms
✓ testFallaPorActividadNula()	1 ms
✓ testFallaPorEdadVacía()	1 ms
✓ testFallaPorVisitanteTotalmen	1 ms
✓ testFallaPorCantidadPersonas	1 ms
✓ testFallaPorNombreConSoloE	1 ms
✓ testFallaPorExcederCupoDispo	1 ms
✓ testFallaPorFaltaDeTalleCuanc	2 ms
✓ testFallaPorTalleInvalido()	1 ms
✓ testFallaPorVisitanteYalnscri	3 ms
✓ testFallaPorHorarioVacio()	1 ms
✓ testFallaPorDniAlfanumerico()	1 ms
✓ testFallaPorParqueCerrado()	2 ms
✓ testFallaPorTalleConCaractere	1 ms
✓ testFallaPorDniVacio()	1 ms
✓ testFallaPorNombreVacio()	1 ms
✓ testFallaPorDniConSoloE	

✓ 30 tests passed 30 tests total, 52 ms

Ejemplo de REFACTOR para casos exitosos:



Ejemplo de REFACTOR para casos fallidos:

